

Bài 4: PHƯƠNG THỨC TRONG LỚP (Số tiết: 5 tiết)

Mục tiêu

- Hiểu được các loại phương thức trong lớp
- Vận dụng gọi được phương thức của đối tượng trong lớp.
- Phân tích, xác định được các lớp, quan hệ giữa các lớp và các thành phần trong lớp cho bài toán cụ thể.

Nội dung chính

- Khái niệm phương thức
- Các loại phương thức trong lớp
- Xác định các phương thức cho lớp
- Cú pháp định nghĩa các phương thức trong lớp
- Gọi phương thức & Nguyên lý truyền thông điệp

4.1 Khái niệm về phương thức

Phương thức là một thành phần quan trọng trong lập trình hướng đối tượng (OOP). Phương thức là một nhóm các câu lệnh được định nghĩa bên trong một lớp, dùng để thao tác trên các thuộc tính của đối tượng.

Phương thức giúp chia nhỏ vai trò của các đối tượng của lớp thành các phần tự độc lập, giúp dễ bảo trì và tái sử dụng.

Hành vi của các đối tượng thuộc một lớp được xác định bởi các **phương thức** của lớp, hành vi thể hiện “vai trò/trách nhiệm” của đối tượng trong hệ thống.

Phương thức chính là các hàm, thủ tục thực hiện các phép xử lý trên các thuộc tính của lớp.

4.2 Các loại phương thức trong lớp

Trong Java, lớp đối tượng có thể chứa nhiều loại phương thức, bao gồm:

- **Phương thức Khởi tạo (Constructor):** Được sử dụng để khởi tạo các đối tượng của lớp. Constructor có cùng tên với lớp và không có kiểu trả về.
- **Phương thức Thực thể (Instance Methods):** hoạt động trên các đối tượng của lớp. Chúng có thể truy cập và thay đổi trạng thái của đối tượng.

- **Phương thức Tĩnh (Static Methods):** Là các phương thức không phụ thuộc vào đối tượng và có thể được gọi mà không cần tạo đối tượng của lớp. Chúng thường dùng để thực hiện các thao tác xử lý trên các thuộc tính static của lớp.
- **Phương thức Trừu tượng (Abstract Methods):** Được khai báo trong lớp trừu tượng và không có phần thân. Các lớp con phải cài đặt nội dung các phương thức này.
- **Phương thức Override (Override Methods):** Phương thức trong lớp con có cùng tên, kiểu trả về và danh sách tham số với phương thức trong lớp cha. Nó thay thế phương thức của lớp cha.
- Phương thức **Overloading (Overloading Methods):** Tạo ra nhiều phương thức trùng tên, khác danh sách các tham số.
- Phương thức **thông thường:** thực hiện các hành vi của đối tượng thuộc lớp, Ví dụ: xét chương trình Java chứa các lớp động vật với thành phần lớp là các phương thức thuộc loại khác nhau như sau:

```
abstract class Animal {
    private String name;

    public Animal(String name) { // Phương thức khởi tạo
        this.name = name;
    }

    public void setName(String name) { // Phương thức thực thể
        this.name = name;
    }

    public String getName() { // Phương thức thực thể
        return name;
    }

    public abstract void makeSound(); // Phương thức trừu tượng
    public void displayInfo() { // Phương thức nạp chồng
        System.out.println("Name: " + name);
    }

    public void displayInfo(boolean showSound) { // Phương thức nạp chồng
        System.out.println("Name: " + name);
    }
}
```

```

        if (showSound) {            makeSound();        }
    }
}

// Lớp con kế thừa từ Animal
class Dog extends Animal {
    // Phương thức khởi tạo (Constructor)
    public Dog(String name) {
        super(name); // Gọi phương thức khởi tạo của lớp cha
    }

    // Phương thức viết đè (Overridden Method)
    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }

    // Phương thức nạp chồng (Overloaded Method) trong lớp con
    public void displayInfo(String extraInfo) {
        super.displayInfo();
        System.out.println("Extra Info: " + extraInfo);
    }

    // Phương thức tĩnh (Static Method) trong lớp con
    public static void printGreeting() {
        System.out.println("Hello from the Dog class!");
    }
}

public class Main {
    public static void main(String[] args) {
        // Gọi phương thức tĩnh từ lớp con
        Dog.printGreeting();

        // Tạo đối tượng của lớp Dog
        Dog myDog = new Dog("Buddy");

        // Gọi phương thức thông thường
        myDog.displayInfo();
    }
}

```

```

    myDog.displayInfo(true);
    // Gọi phương thức thực thể
    myDog.setName("Jenny");
    // Gọi phương thức nạp chồng
    myDog.displayInfo("Loves to play fetch!");
    // Gọi phương thức viết đề
    myDog.makeSound();
}
}

```

4.3 Xác định các phương thức cho lớp

Ví dụ: xét bài toán “Quản lý sinh viên” cho một trường đại học:

Một trường đại học cần một hệ thống để quản lý thông tin sinh viên. Hệ thống này phải lưu trữ thông tin cá nhân của sinh viên, điểm số của từng môn học và tính điểm trung bình cho mỗi sinh viên.

Yêu cầu: phân tích đề

- Xác định các lớp
- Mối quan hệ giữa các lớp
- Xác định các thuộc tính và phương thức cho mỗi lớp
- Cài đặt chương trình

Lời giải:

Xác định các lớp

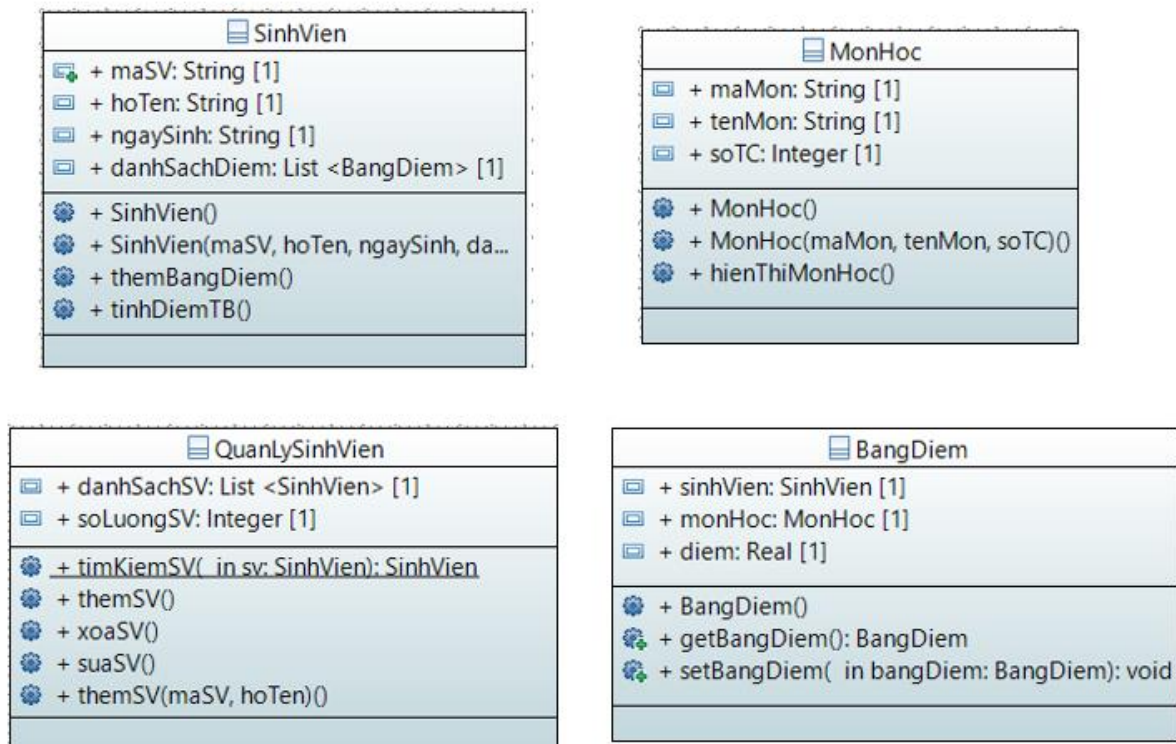
- SinhVien: Đại diện cho mỗi sinh viên, chứa các thuộc tính như tên, mã sinh viên, ngày sinh, ... và một danh sách các môn học.
- MonHoc: Đại diện cho mỗi môn học, chứa các thuộc tính như tên môn, mã môn, và điểm số, ...
- Bảng điểm: Quản lý điểm số của sinh viên
- QLSinhVien: quản lý danh sách các SV trong trường, hoặc quản lý danh sách các lớp học trong trường.

- Có thể bổ sung các lớp khác như: Học kỳ (một học kỳ có nhiều môn học), Lớp học.

Mối quan hệ giữa các lớp?

- Lớp sinh viên có một danh sách bảng điểm (một sinh viên học nhiều môn)
- Lớp bảng điểm liên kết với môn học để xác định từng điểm số là của môn học nào
- Lớp quản lý sinh viên đóng vai trò quản lý tập hợp các sinh viên
- ...

Xác định các thuộc tính và phương thức cho từng lớp?

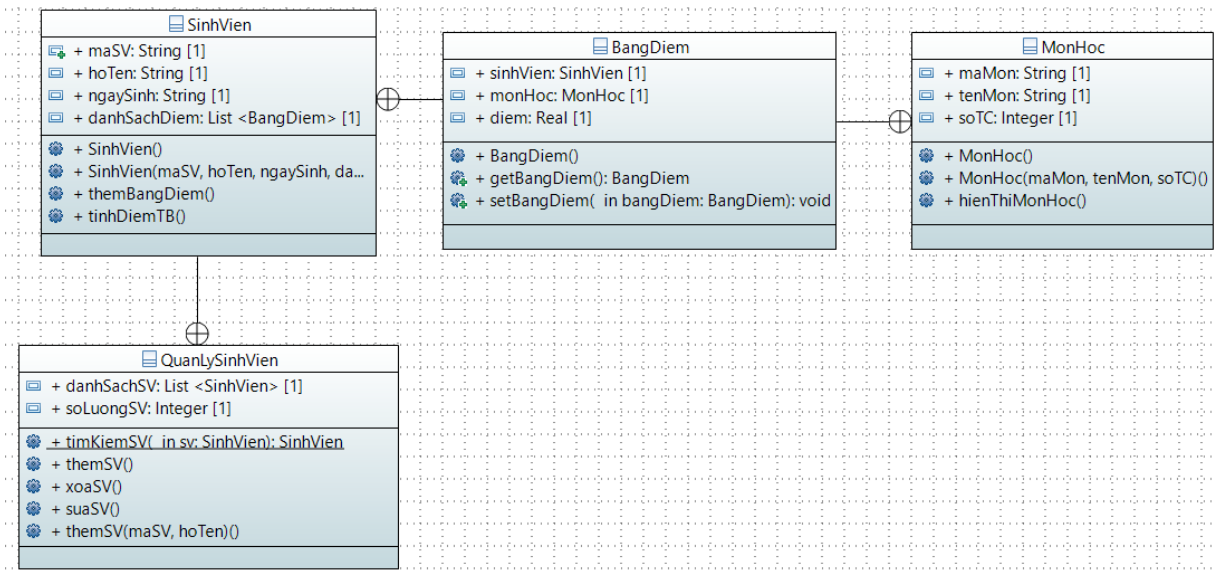


Hình 4.1 Danh sách các lớp cho bài toán quản lý điểm Sinh viên

Gắn quan hệ giữa các lớp

=> Lớp QuanLySV chứa nhiều SinhVien, một sinh viên học nhiều môn học, một môn học được học bởi nhiều sinh viên, chuẩn hóa quan hệ m-n này thành bảng trung gian: Bảng điểm, quan hệ sau khi chuẩn hóa là: một sinh viên có

nhiều bảng điểm, mỗi bảng điểm cho một môn học; một môn học có nhiều bảng điểm, mỗi bảng điểm cho một sinh viên.



Hình 4.2 Quan hệ giữa các lớp

Triển khai:

- Xây dựng các lớp: mã nguồn các lớp?
- Mở rộng chương trình?
- ❖ Áp dụng nguyên lý Encapsulation
 - => Tạo thêm các phương thức getter, setter
- ❖ Áp dụng nguyên lý kế thừa
 - => Tạo thêm lớp SinhVienKhoaCNTT kế thừa từ SinhVien
 - => Nạp viết đè, nạp chồng các phương thức?
- ❖ Áp dụng nguyên lý trừu tượng hóa
 - => Biến lớp SinhVien thành lớp trừu tượng
 - ⇒ Các phương thức trừu tượng của lớp?
 - ⇒ Triển khai viết đè các phương thức trừu tượng?
- ❖ Áp dụng nguyên lý đa hình

⇒ Quản lý các đối tượng như thế nào, đa hình trong hành vi, đa hình thái xuất hiện.

Mã nguồn chương trình mở rộng?

4.4 Cú pháp định nghĩa các loại phương thức

Cú pháp chung:

<Phạm_vi> <bổ ngữ> <Kiểu_giá_tri> <tên_hàm> (<danh sách các tham số hình thức>) <các_mệnh_đề_throws> { // Thân hàm }

Phạm_vi:

- *public*: Cho phép truy cập mọi nơi trong hệ thống
- *private*: Chỉ được truy xuất (gọi) bên trong lớp chứa nó
- *protected*: lớp đó và lớp con của nó được truy xuất
- *final*: Không cho phép nạp chồng và ghi đè
- *static*: Nó có ý nghĩa tương tự như với thuộc tính *static*, thông thường chỉ những phương thức chung và không dùng cho một đối tượng cụ thể thì khi đó phương thức đó nó mới là phương thức tĩnh (*static*).
- *abstract*: Phương thức trừu tượng (là phương thức “mẫu”, nó chỉ nói lên là nó làm được gì mà không mô tả nó làm như thế nào), do đó nó không cài đặt nội dung ở lớp khai báo nó
- *native*: dùng cho phương thức khi cài đặt phụ thuộc môi trường trong một ngôn ngữ khác, như C hay hợp ngữ.
- *synchronized*: dùng trong lập trình đa luồng, nhằm ngăn các tác động của các phương thức khác của các luồng khác khi phương thức đang được thực hiện.
- Giá trị mặc định là *public*.

<Kiểu trả lại> có thể là kiểu nguyên thủy, kiểu lớp hoặc không có giá trị trả lại (kiểu *void*).

<Danh sách tham biến hình thức> bao gồm dãy các tham biến (kiểu và tên) phân cách với nhau bởi dấu phẩy.

Trong đó:

- Tùy theo *<bổ ngữ>* là gì ta sẽ có các loại phương thức: *static*, *abstract*
- Tên phương thức bắt đầu bằng *set<tenThuocTinh>()*, *get<tenThuocTinh>()* sẽ là các phương thức thực thể

Cú pháp khai báo constructors:

Khai báo: [*<phạm vi>*] *<Tên lớp>* ([*<danh sách tham biến>*]) {

// Nội dung cần tạo lập

}

Trong đó:

- Phạm_vì có thể là: public, protected, private
- Phần thân là các lệnh khởi tạo giá trị các thuộc tính của đối tượng, là thành phần tùy chọn.
- Danh sách các tham số : tùy chọn

Phân loại:

- Toán tử tạo lập mặc định không tường minh: **Tên_Lớp()** {}. Khi định nghĩa một lớp mà không xây dựng toán tử tạo lập nào thì hệ thống sẽ cung cấp toán tử tạo lập mặc định không tường minh.
- Toán tử tạo lập mặc định tường minh:

```
TênLớp(<danh sách tham biến hình thức>) {  
    tên thuộc tính= giá trị;  
    ..... }
```

Giá trị của các biến thành phần được khởi tạo không mặc định bằng các phép gán trong thân của toán tử bằng các giá trị được truyền vào.

Ví dụ 5: Minh họa một phương thức khởi tạo của lớp Person bằng cách gán giá trị cho các thuộc tính tên và tuổi.

```
class Person {  
    public String name;  
    public int age;  
    // Phương thức tạo lập mặc định không tường minh  
    // public Person(){}  
    // Phương thức tạo lập mặc định tường minh  
    public Person(){  
        name = "";  
        age = 16;  
    }  
    // Phương thức tạo lập không mặc định  
    public Person(String name1, int age1){  
        name = name1;  
        age = age1;  
    }  
}
```



```

    }

    // Phương thức tạo lập không mặc định
    public Person(String name1, int age1){
        name = name1;
        age = age1;
    }

    /*public Person(String name, int age){
this.name = name;
this.age = age;
}*/

    public void show(){
        System.out.println( name + “ is ” + age + “ years old!”);
    }
}

```

Nạp chồng các toán tử tạo lập

Giống như các hàm thành phần, các toán tử tạo lập có thể nạp chồng với nhiều nội dung thực hiện khác nhau.

Khi tạo lập đối tượng sử dụng toán tử tạo lập nào thì toán tử đó được gọi.

Ví dụ 6:

```

class BongDen{
    // Biến thành phần (1)
    private int soWatts;
    private boolean batTat;
    private String viTri;
    // Định nghĩa toán tử tạo lập số 1 (2)
    BongDen(){
        this(40, true);
        System.out.println(“Toán tử số 1”); }
    // Định nghĩa toán tử tạo lập số 2 (3)
    BongDen(int w, boolean s){

```

```

        this(w, s, "XX");

        System.out.println("Toán tử số 2"); }

//Định nghĩa toán tử tạo lập số 3 (4)

BongDen(int soWatts, boolean batTat, String viTri){

    this.soWatts = soWatts;

    this.batTat = batTat;

    this.viTri = new String(viTri);

    System.out.println("Toán tử số 3"); }

}

class DenTuyp extends BongDen {

    private int doDai;

    private int mau;

    DenTuyp(int leng, int colo){

        this(leng,colo,100,true,"Chua biet");

    }

    DenTuyp(int leng, int colo, int soWatt, boolean bt, String noi){

        super(soWatt, bt, noi);

        this.doDai = leng;

        this.mau = colo;

    }

}

public class NhaKho{

    public static void main(String args[]){

        System.out.println("Tao ra bong đèn tuyp");

        DenTuyp d = new DenTuyp(20, 5);

    }

}

```

4.5 Gọi phương thức và nguyên lý truyền thông điệp

Truyền các giá trị kiểu nguyên thủy

Trong Java, khi gọi một phương thức và truyền các giá trị nguyên thủy cho phương thức, được gọi là truyền tham trị. Việc thay đổi giá trị chỉ có hiệu lực trong phương thức được gọi, không có hiệu lực bên ngoài phương thức.

Các tham số có kiểu dữ liệu nguyên thủy là byte, short, int, long, float, double, boolean, char.

Ví dụ 3: *Truyền các giá trị nguyên thủy*

```
class KháchHang1 { // Lớp khách hàng

    public static void main(String[] arg) {

        HangSX banh = new HangSX(); // Tạo ra một đối tượng

        int giaBan = 20;

        double tien = banh.tinh(10, giaBan);

        System.out.println("Gia ban: " + giaBan); // giaBan không đổi

        System.out.println("Tien ban duoc : " + tien);

    }

}

// Lớp Hãng sản xuất

class HangSX {

    double tinh(int num, double gia) {

        gia = gia / 2;

        return num * gia; // Thay đổi gia nhưng không ảnh hưởng tới
        giaBan, số tiền bị thay đổi theo

    }

}
```

Truyền các giá trị tham chiếu đối tượng.

Trong Java, khi gọi một phương thức và truyền một tham chiếu cho phương thức, được gọi là truyền tham chiếu. Việc thay đổi giá trị của biến tham chiếu bên trong phương thức làm thay đổi giá trị của nó.

Trong Java, tất các phương thức có tham số là biến có kiểu là các lớp (class) đều là kiểu tham chiếu

//KhachHang2.java

```
class KháchHang2 { // Lớp khách hàng

    public static void main(String[] arg) {
```

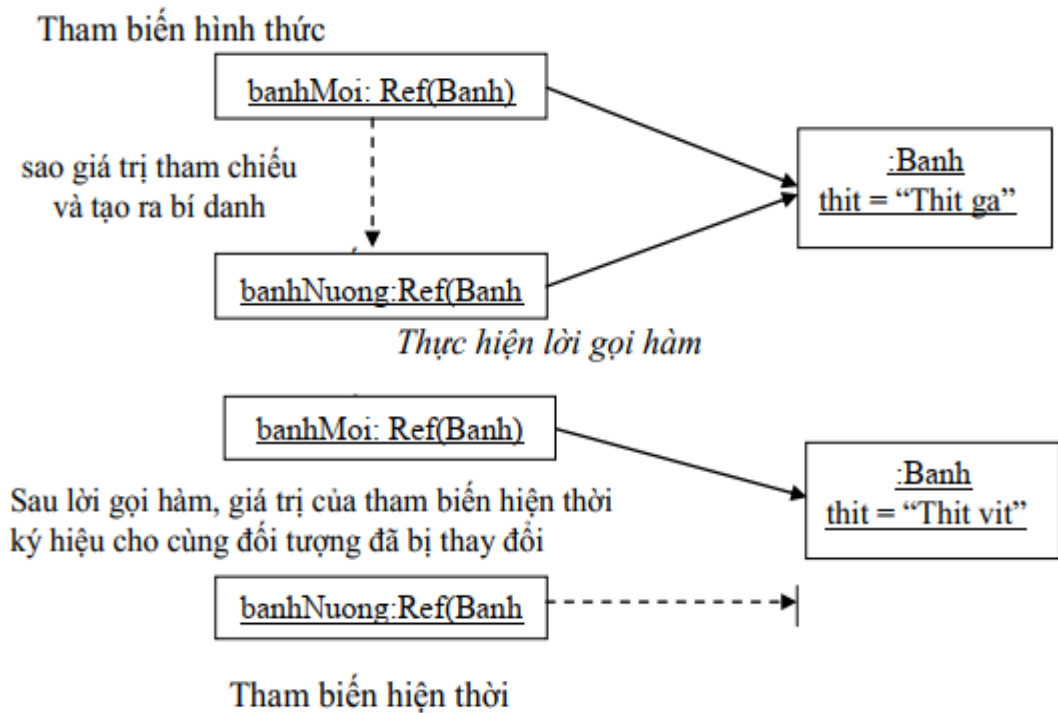
```

        Banh banhMoi = new Banh(); // Tạo ra một đối tượng (1)
        System.out.println("Nhoi thit vao banh truoac khi nuong:" +
        banhMoi.thit);
        nuong(banhMoi); // (2)
        System.out.println("Thit cua banh sau khi nuong:" +
        banhMoi.thit);
    }

    public static void nuong(Banh banhNuong) { // (3)
        banhNuong.thit = "Thit vit
        "; // đổi nhân thành thịt vịt
        banhNuong = null; // (4)
    }
}

class Banh { // Lớp Banh (5)
    String thit = "Thit ga
    "; // Quy định của hãng làm nhân bánh bằng thịt gà
}

```



Hình 4 Error! No text of specified style in document..1 Truyền các giá trị tham chiếu đối tượng

Truyền tham chiếu: đối với đối tượng thì nội dung của tham chiếu (LValue) được copy lên stack.

Truyền đối số trong dòng lệnh

```
class Pass {  
    public static void main(String parameters[]) {  
        System.out.println("This is what the main method received");  
        System.out.println(parameters[0]);  
        System.out.println(parameters[1]);  
        System.out.println(parameters[2]);  
    }  
}
```

Biên dịch chương trình: **javac PassArgument.java**

Thực thi chương trình với dòng lệnh: **java PassArgument A 123 B1**

Sẽ thu được trên màn hình kết quả:

```
This is what the main method received  
A  
123  
B1
```

Các tham biến final

Tham biến loại này được gọi là biến cuối “trắng”, nghĩa là nó không được khởi tạo giá trị (là trắng) cho đến khi nó được gán một trị nào đó và khi đã được gán trị thì giá trị đó là cuối cùng, không thay đổi được.

Ví dụ 4: Sử dụng tham biến final

//KhachHang3.java

```
class KhachHang3 { // Lớp khách hàng  
    public static void main(String[] arg) {  
        HangSX banh = new HangSX(); // Tạo ra 1 đối tượng  
        int giaBan = 20;  
        double tien = banh.tinh(10, giaBan);  
        System.out.println("Gia ban: " + giaBan); // giaBan không đổi
```

```

        System.out.println("Tien ban duoc : " + tien);
    }
}

// Lớp Hãng sản xuất
class HangSX {
    double tinh(int num, final double gia) { // (1)
        gia = gia / 2.0; // (2)
        return num * gia; // Thay đổi gia nhưng không ảnh hưởng tới giaBan, số
        tiền vẫn bị thay đổi theo
    }
}

```

Tổng kết bài học

Bài học này cung cấp kiến thức các kiến thức cơ bản về các loại phương thức có thể có trong phạm vi một lớp. Ngoài ra cũng trình bày sơ lược quy trình đi từ bài toán đến chương trình Java theo hướng đối tượng: xác định các lớp, quan hệ giữa các lớp, các thuộc tính và phương thức của lớp, lập trình mã hóa giải pháp thiết kế, tối ưu giải pháp thiết kế bằng cách sử dụng các nguyên lý Encapsulation, Inheritance, Abstraction, và Polymorphism.

Bài tập thực hành

Bài tập 1:

Trường ĐH X muốn xây dựng chương trình Java quản lý sinh viên, giảng viên, nhân viên và các khóa học sinh viên đăng ký trong một kỳ học.

Yêu cầu:

- 1) Phân tích xác định các lớp đối tượng cần quản lý (Người, Giảng viên, Sinh viên, Khóa học, Kỳ học)
- 2) Phân tích xác định quan hệ giữa các lớp
- 3) Phân tích xác định các thuộc tính, phương thức của lớp
- 4) Cài đặt chương trình Java định nghĩa các lớp đã xác định.

Bài tập 2: Quản lý tài khoản ngân hàng

Mô tả: Ngân hàng cần một hệ thống để quản lý tài khoản của khách hàng, bao gồm việc mở tài khoản, nạp tiền, rút tiền, và kiểm tra số dư. Mỗi khách hàng có thể có nhiều tài khoản với số dư khác nhau.

Gợi ý giải quyết:

- **Lớp:** BankAccount (Tài khoản ngân hàng), Customer (Khách hàng).
- **Thuộc tính:** accountNumber, balance, accountType cho BankAccount; customerID, name, accounts cho Customer.
- **Phương thức:** deposit(), withdraw(), checkBalance() cho BankAccount; addAccount() cho Customer.

Bài tập 3: Quản lý cửa hàng sách

Mô tả: Một cửa hàng sách cần hệ thống để quản lý sách, tác giả, và đơn hàng. Hệ thống cần lưu trữ thông tin sách, theo dõi số lượng sách bán ra, và quản lý đơn hàng của khách hàng.

Gợi ý giải quyết:

- **Lớp:** Book (Sách), Author (Tác giả), Order (Đơn hàng).
- **Thuộc tính:** bookID, title, author, price, stock cho Book; authorID, name, booksWritten cho Author; orderID, customer, booksOrdered cho Order.
- **Phương thức:** sellBook(), updateStock() cho Book; addBook() cho Author; calculateTotalPrice() cho Order.